Structured Query Language

02 August 2026 08:45

There are many relational database management system(RDBMS) such as MySQL, Microsoft SQL Server, PostgreSQL, Oracle, etc. that allow as to create a database consisting of relations. These RDBMS also allow us to store, retrieve and manipulate data on that database through queries.

Structured Query Language (SQL)
One has to write application programs to access data in case of a file system. However, for database management system, there are a special kind of languages called Query language that can be used to access and manipulate data from the database. The Structured Query Language (SQL) is the most popular query language used by major relational database management systems such as MySQL, Oracle, SQL Server, etc.

SQL is easy to learn as the statements comprise of descriptive English words and are not case sensitive. We can create and interact with a database using SQL easily. Benefit of using SQL is that we do not have to specify how to get the data from the database. Rather, we simply specify what is to be retrieved and SQL does the rest. Although called a query language, SQL can do much more beside querying. SQL provides statements for defining the structure of the data, manipulating data in the database, declaring constraints, and retrieving data from the database in various ways depending on our requirements.

Data types and constraints in MySQL
We know that data. Consist of one or more relations and each relation (table) is made up of attributes (column) Each attribute has a data type. We can also specify constraints for each attribute of a relation.

Data type of attribute
Data type of an attribute indicates the type of data value that an attribute can have. It also decides the operations that can be performed on the data. Of that attribute. For example, arithmetic operations can be performed on numeric data, but not on character data. Commonly used data types in MySQL are numeric types, date and time types, and string types.

Commonly used data types in MySQL

Data type	Description
CHAR(n)	Specifies character type data of length N, where N could be any value from zero to 255. CHAR is of fixed length means declaring CHAR(10) implies to reserve spaces for ten characters. If data does not have 10 characters (Example city has four characters) MySQL fills the remaining six characters with spaces padded on the right.
VARCHAR(n)	The specifies character type data of length where N could be any value from zero to 65,535. But unlike CHAR VARCHAR (N) is a variable length data type. That is declaring VARCHAR (30) means a

	maximum of 30 characters can be stored, but the actual allocated bytes will depend on the length of entering string. So 'city' in VARCHAR(30) will occupy space needed to store four characters only.
INT	Int specifies an integer value. Each int value occupies four bytes of storage. The range of unassigned values allowed in a four byte integer type are 0 to 4294967295. For values larger than that we have to use BIGINT which occupies eight bytes.
FLOAT	Holds number with decimal points. Each float value occupies four bytes.
DATE	The DATE type is used for dates in 'YYYY-MM-DD' Format. YYYY is the 4-digit year, MM is the 2 digit month and DD is the 2-digit day. The supporting range is '1000-01-01' to '9999-12-31'.

Constraints: Constraints are the certain types of restrictions on the data values that an attribute can have. They are used to ensure correctness of data. However, it is not mandatory to define constraints for each attribute of a table.

Commonly used constraints

Constraints	Description
NOT NULL	Ensures that the column cannot have null values, where NULL means. Missing unknown or not applicable values.
UNIQUE	Ensures that all the values in a column are distinct or unique.
DEFAULT	A default value specified for the column if no value is provided.
PRIMARY KEY	The column which can uniquely identify each row or record in a table.
FOREIGN KEY	The column which refers to a value of an attribute defined as a primary key in another table.

SQL for data definition

In order to be able to store data, we need to first define the relation schema. Defining a schema includes. Creating a relation and giving name to relation, identifying the attributes in a relation, deciding upon the data type for each attribute and also specify the constraints as per requirements. Sometimes we may require to make changes to the relation schema also. SQL allows us to write statements for defining, modifying and deleting relation schemas. These are part of Data Definition Language (DDL).

We have already learned that the data are stored in relations or tables in a database. Hence we can say that database is a collection of tables. The CREATE statement is used to create a database and its table (relations) Before creating a database we should be clear about the number of tables and database will have the. Columns (attributes) in each table, along with the data type of each column and its constraints, if any.

Create Database

To create a database we use the CREATE DATABASE statement as shown in the following

syntax:

```
CREATE DATABASE databasename;
```

To create a database, calls. Student attendance we will type the following command at MySQL prompt.

A RDBMS can manage multiple databases on 1 computer. Therefore, we need to select the database that we want to use. To know the names of existing databases, we use the statement SHOW DATABASES. From the listed databases we can select the database to be used. Once the database is selected, we can proceed with creating tables or querying data.
In order to use the student attendance database, the following SQL statement is required.

```
mysql> USE student_attendance;
Database changed
```

Initially the created database is empty. It can be checked by using the SHOW TABLES statement that lists names of all the tables within the database.

```
mysql> SHOW TABLES;
Empty set (0.027 sec)
```

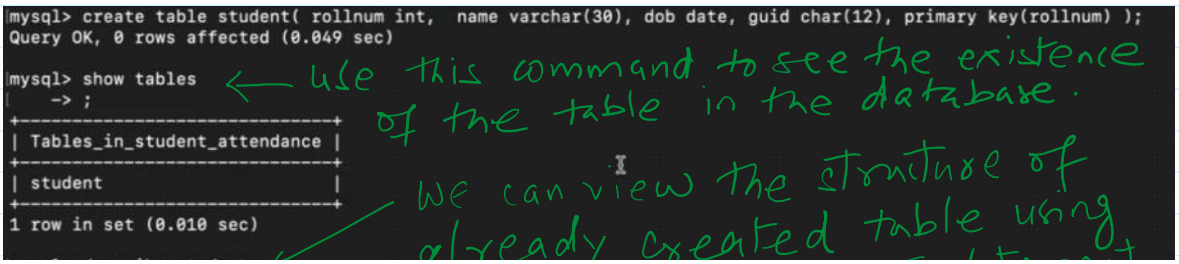
Create table

After creating a database student _attendance, we need to define relations in this database and specify attributes for each relation along with data type and constraints(if any) for each attribute. This is done using the CREATE TABLE statement.
Syntax:

```
CREATE TABLE tablename(
   Attributename1 datatype constraint,
   Attributename2 datatype constraint,
   Attributename3 datatype constraint,
   …
   Attributenamen datatype constraint
   );
```

Important to observe the following points with respect to CREATE TABLE statement:

- The number of columns in a table defines the degree of that relation, which is denoted by N.
- Attribute name specifies the name of the column in the table.
- Data type species. Type of data that an attribute can hold.
- Constraints indicate the restriction imposed on the value of an attribute. By default, each attribute can take null values except for the primary key.



```
| student                      |
+------------------------------+
1 row in set (0.010 sec)

mysql> describe student;
+---------+-------------+------+-----+---------+-------+
| Field   | Type        | Null | Key | Default | Extra |
+---------+-------------+------+-----+---------+-------+
| rollnum | int         | NO   | PRI | NULL    |       |
| name    | varchar(30) | YES  |     | NULL    |       |
| dob     | date        | YES  |     | NULL    |       |
| guid    | char(12)    | YES  |     | NULL    |       |
+---------+-------------+------+-----+---------+-------+
4 rows in set (0.022 sec)
```

We can view the structure of already created table using DESCRIBE statement.

ALTER Table

After creating a table, we may realize that we need to add/remove an attribute or to modify the data type of an existing attribute or to add constraints in attribute. In all such cases we need to change or alter the structure (schema) of the table by using the alter statement.

Add primary key to a relation

Let us now alter the table student. The following MySQL statement adds a primary key to the student table:

```
mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | YES  |     | NULL    |       |
| name        | varchar(30) | YES  |     | NULL    |       |
| guid        | char(10)    | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
3 rows in set (0.244 sec)

mysql> ALTER TABLE student ADD PRIMARY KEY(roll_number);
Query OK, 0 rows affected (0.577 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(30) | YES  |     | NULL    |       |
| guid        | char(10)    | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
3 rows in set (0.052 sec)
```

```
mysql> ALTER TABLE guardian ADD PRIMARY KEY(guid);
Query OK, 0 rows affected (0.464 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc guardian
    -> ;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | int         | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  |     | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.041 sec)
```

```
mysql> ALTER TABLE attendance ADD PRIMARY KEY(attendance_date, roll_number);
Query OK, 0 rows affected (0.413 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc attendance;
+-------------------+---------+------+-----+---------+-------+
| Field             | Type    | Null | Key | Default | Extra |
+-------------------+---------+------+-----+---------+-------+
| attendance_date   | date    | NO   | PRI | NULL    |       |
| roll_number       | int     | NO   | PRI | NULL    |       |
| attendance_status | char(1) | YES  |     | NULL    |       |
+-------------------+---------+------+-----+---------+-------+
3 rows in set (0.050 sec)
```

Add foreign key to a relation

Once primary keys are added, the next step. Is to add foreign key to the relation (if any) Following points needed to be observed while adding foreign key to a relation:

- The reference to relation must be already created.
- The referenced attribute(s) Must be the part of the primary key of the referenced relation.
- Data types and size of the referenced and referencing attributes must be the same.

```
mysql> desc attendance;
+-------------------+---------+------+-----+---------+-------+
| Field             | Type    | Null | Key | Default | Extra |
+-------------------+---------+------+-----+---------+-------+
| attendance_date   | date    | NO   | PRI | NULL    |       |
| roll_number  FK   | int     | NO   | PRI | NULL    |       |
| attendance_status | char(1) | YES  |     | NULL    |       |
+-------------------+---------+------+-----+---------+-------+
3 rows in set (0.048 sec)
```

```
mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(30) | YES  |     | NULL    |       |
| guid   FK   | char(10)    | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
3 rows in set (0.056 sec)

mysql> desc guardian
    -> ;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | int         | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  |     | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.028 sec)
```

Syntax:

```
ALTER TABLE table_name ADD FOREIGN KEY(attribute_name) referenced
Table_name(attribute_name);
```

Let us now add foreign key to the table Student. The attribute GUID (The referencing attribute) is a foreign key, and it refers to the attribute GUID. (the referenced attribute) of the table guardian. Hence Student is the referencing table and Guardian is the reference table.

```
mysql> ALTER TABLE student
    -> ADD FOREIGN KEY(guid) REFERENCES guardian(guid);
Query OK, 0 rows affected (0.843 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

```
mysql> ALTER TABLE attendance ADD FOREIGN KEY(roll_number) REFERENCES student(roll_number);
Query OK, 0 rows affected (0.770 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc attendance;
+-------------------+---------+------+-----+---------+-------+
| Field             | Type    | Null | Key | Default | Extra |
+-------------------+---------+------+-----+---------+-------+
| attendance_date   | date    | NO   | PRI | NULL    |       |
| roll_number       | int     | NO   | PRI | NULL    |       |
| attendance_status | char(1) | YES  |     | NULL    |       |
+-------------------+---------+------+-----+---------+-------+
3 rows in set (0.028 sec)
```

Modify datatype of an attribute:

Syntax:

```
ALTER TABLE table_name MODIFY attribute_name DATATYPE;
```

For example:

```
mysql> ALTER TABLE guardian MODIFY guid CHAR(10);
Query OK, 0 rows affected (1.140 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Add constraint UNIQUE to an existing attribute

In guardian table, the attribute mobile_number has a constraint unique which means no two values in that column should be same.

Syntax:

```
ALTER TABLE table_name ADD UNIQUE(attribute_name);
```

```
mysql> desc guardian;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | char(10)    | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  |     | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.048 sec)

mysql> ALTER TABLE guardian ADD UNIQUE(mobile_number);
Query OK, 0 rows affected (0.346 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc guardian;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | char(10)    | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  | UNI | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.036 sec)
```

Add an attribute to an existing table

Sometimes we may need to add an additional attribute in a table. It can be done using the add attribute statement as shown in the following syntax :

Syntax:

```
ALTER TABLE table_name ADD attribute_name DATATYPE;
```

```
mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(30) | YES  |     | NULL    |       |
| guid        | char(10)    | YES  | MUL | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
3 rows in set (0.051 sec)

mysql> ALTER TABLE student ADD dob DATE;
Query OK, 0 rows affected (0.677 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(30) | YES  |     | NULL    |       |
| guid        | char(10)    | YES  | MUL | NULL    |       |
| dob         | date        | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
4 rows in set (0.049 sec)
```

Modify constraint of an attribute

When we. By default each attribute takes null value except for the attribute defined as primary key. We can change an attributes constraint from NULL to NOT NULL using an ALTER statement.

Syntax:

```
ALTER TABLE table_name MODIFY attribute DATATYPE NOT NULL;
```

Note: We have to specify the data type of the attribute along with the constraint not null while using modify.

```
mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(30) | YES  |     | NULL    |       |
| guid        | char(10)    | YES  | MUL | NULL    |       |
| dob         | date        | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
4 rows in set (0.049 sec)

mysql> ALTER TABLE student MODIFY name VARCHAR(50) NOT NULL;
Query OK, 0 rows affected (0.596 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(50) | NO   |     | NULL    |       |
| guid        | char(10)    | YES  | MUL | NULL    |       |
| dob         | date        | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
4 rows in set (0.059 sec)
```

Add default value to an attribute

If we want to specify default value for an attribute then use the following syntax:

```
ALTER TABLE table_name
MODIFY attribute_name DATATYPE
DEFAULT default_value;
```

```
mysql> desc student;
+-------------+-------------+------+-----+---------+-------+
| Field       | Type        | Null | Key | Default | Extra |
+-------------+-------------+------+-----+---------+-------+
| roll_number | int         | NO   | PRI | NULL    |       |
| name        | varchar(50) | NO   |     | NULL    |       |
| guid        | char(10)    | YES  | MUL | NULL    |       |
| dob         | date        | YES  |     | NULL    |       |
+-------------+-------------+------+-----+---------+-------+
4 rows in set (0.059 sec)

mysql> ALTER TABLE student
    -> MODIFY dob DATE
    -> DEFAULT '2000-05-15';
Query OK, 0 rows affected (0.753 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc student;
+-------------+-------------+------+-----+------------+-------+
| Field       | Type        | Null | Key | Default    | Extra |
+-------------+-------------+------+-----+------------+-------+
| roll_number | int         | NO   | PRI | NULL       |       |
| name        | varchar(50) | NO   |     | NULL       |       |
| guid        | char(10)    | YES  | MUL | NULL       |       |
| dob         | date        | YES  |     | 2000-05-15 |       |
+-------------+-------------+------+-----+------------+-------+
4 rows in set (0.054 sec)
```

Remove an attribute

Using ALTER we can remove attributes from a table as shown in the following syntax:

```
ALTER TABLE table_name DROP attribute;
```

```
mysql> desc guardian;
+---------------+---------------+------+-----+---------+-------+
| Field         | Type          | Null | Key | Default | Extra |
+---------------+---------------+------+-----+---------+-------+
| guid          | char(10)      | NO   | PRI | NULL    |       |
| name          | varchar(30)   | YES  |     | NULL    |       |
| mobile_number | varchar(10)   | YES  | UNI | NULL    |       |
| address       | varchar(50)   | YES  |     | NULL    |       |
| income        | decimal(10,2) | YES  |     | NULL    |       |
+---------------+---------------+------+-----+---------+-------+
5 rows in set (0.150 sec)

mysql> ALTER TABLE guardian DROP income;
Query OK, 0 rows affected (0.575 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> desc guardian;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | char(10)    | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  | UNI | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.052 sec)
```

Remove PRIMARY KEY from the table

Sometimes there may be requirement to remove the primary key constraint from the table. In that case ALTER TABLE command can be used in the following way:

Syntax:

```
ALTER TABLE table_name DROP PRIMARY KEY;
```

```
mysql> desc guardian;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | char(10)    | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  | UNI | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.052 sec)

mysql> ALTER TABLE guardian DROP PRIMARY KEY;
ERROR 1553 (HY000): Cannot drop index 'PRIMARY': needed in a foreign key constraint
```

Note: We cannot drop the primary key of guardian table because the guid field is referenced in another table or relation, so the data of these table may be affected, so MySQL will not allow us to remove the primary key.
First we need to remove foreign key then we can remove the primary key.

DROP Statement

Sometimes a table in a database or the database itself needs to be removed. We can use a DROP statement to remove a database or a table permanently from the system. However, one should be very cautious while using this statement as it cannot be undone.

Syntax to drop a table:

```
DROP TABLE table_name;
```

Syntax to drop a database:

```
DROP DATABASE databse_name;
```

Note: Using the DROP statement to remove a database will ultimately remove all the tables within it.

For example:

```
mysql> CREATE TABLE t1(
    -> x int,
    -> y int);
Query OK, 0 rows affected (1.393 sec)

mysql> show tables;
+------------------------------+
| Tables_in_student_attendance |
+------------------------------+
| attendance                   |
| guardian                     |
| student                      |
| t1                           |
+------------------------------+
4 rows in set (0.041 sec)

mysql> DROP TABLE t1;
Query OK, 0 rows affected (0.775 sec)

mysql> show tables;
+------------------------------+
| Tables_in_student_attendance |
+------------------------------+
| attendance                   |
| guardian                     |
| student                      |
+------------------------------+
3 rows in set (0.043 sec)
```

SQL for DATA MANIPULATION

We created the database student_attendance having 3 relations student, guardian and attendance. When we create a table, only its structure is created but the table has no data. To populate records in the table, **INSERT** statement as used. Also, table records can be deleted or updated using DELETE and UPDATE statements. These SQL statements are part of Data Manipulation Language (DML).

Data manipulation using a database means either insertion of new record or a data, removal of existing data or modification of existing data in the database.

INSERTION of RECORDS

INSERT INTO Statement is used to insert new records in a table. Its syntax is:

INSERT INTO table_name VALUES(value1, value2,…);

Here value1 corresponds to attribute1, value2 corresponds to attribute2 and so on. Note that we need not to specify attribute names in the insert statement if there are exactly same number of values in the insert statement as the total number of attributes in the table.

```
mysql> desc guardian
    -> ;
+---------------+-------------+------+-----+---------+-------+
| Field         | Type        | Null | Key | Default | Extra |
+---------------+-------------+------+-----+---------+-------+
| guid          | char(10)    | NO   | PRI | NULL    |       |
| name          | varchar(30) | YES  |     | NULL    |       |
| mobile_number | varchar(10) | YES  | UNI | NULL    |       |
| address       | varchar(50) | YES  |     | NULL    |       |
+---------------+-------------+------+-----+---------+-------+
4 rows in set (0.052 sec)

mysql> INSERT INTO guardian VALUES('G000000001', 'Rakesh Jhunjhunwalla', 987678651, 'Salt Lake');
Query OK, 1 row affected (0.829 sec)

mysql> select * from guardian;
+------------+---------------------+---------------+-----------+
| guid       | name                | mobile_number | address   |
+------------+---------------------+---------------+-----------+
| G000000001 | Rakesh Jhunjhunwalla | 987678651     | Salt Lake |
+------------+---------------------+---------------+-----------+
1 row in set (0.026 sec)
```

Caution: While populating records in a table with foreign key, ensure that records in reference table are already populated.

```
mysql> INSERT INTO guardian VALUES('G000000002', 'Mohit Singhi', 987679871, 'Salt Lake');
Query OK, 1 row affected (0.453 sec)

mysql> INSERT INTO guardian VALUES('G000000003', 'Himanshu Agarwal', 987670081, 'Salt Lake');
Query OK, 1 row affected (0.460 sec)

mysql> INSERT INTO guardian VALUES('G000000004', 'Amit Ahuja', 988670081, 'Salt Lake');
Query OK, 1 row affected (0.477 sec)

mysql> INSERT INTO guardian VALUES('G000000005', 'Rajesh Singh', 988670099, 'Salt Lake');
Query OK, 1 row affected (0.450 sec)

mysql> select * from guardian;
+------------+---------------------+---------------+-----------+
| guid       | name                | mobile_number | address   |
+------------+---------------------+---------------+-----------+
| G000000001 | Rakesh Jhunjhunwalla | 987678651     | Salt Lake |
| G000000002 | Mohit Singhi        | 987679871     | Salt Lake |
| G000000003 | Himanshu Agarwal    | 987670081     | Salt Lake |
| G000000004 | Amit Ahuja          | 988670081     | Salt Lake |
| G000000005 | Rajesh Singh        | 988670099     | Salt Lake |
+------------+---------------------+---------------+-----------+
5 rows in set (0.299 sec)
```

If we want to insert values only for some of the attributes in a table (supposing other attributes having null or any other default value), Then we shall specify the attribute names in which the values are to be inserted using the following syntax of INSERT INTO statement.

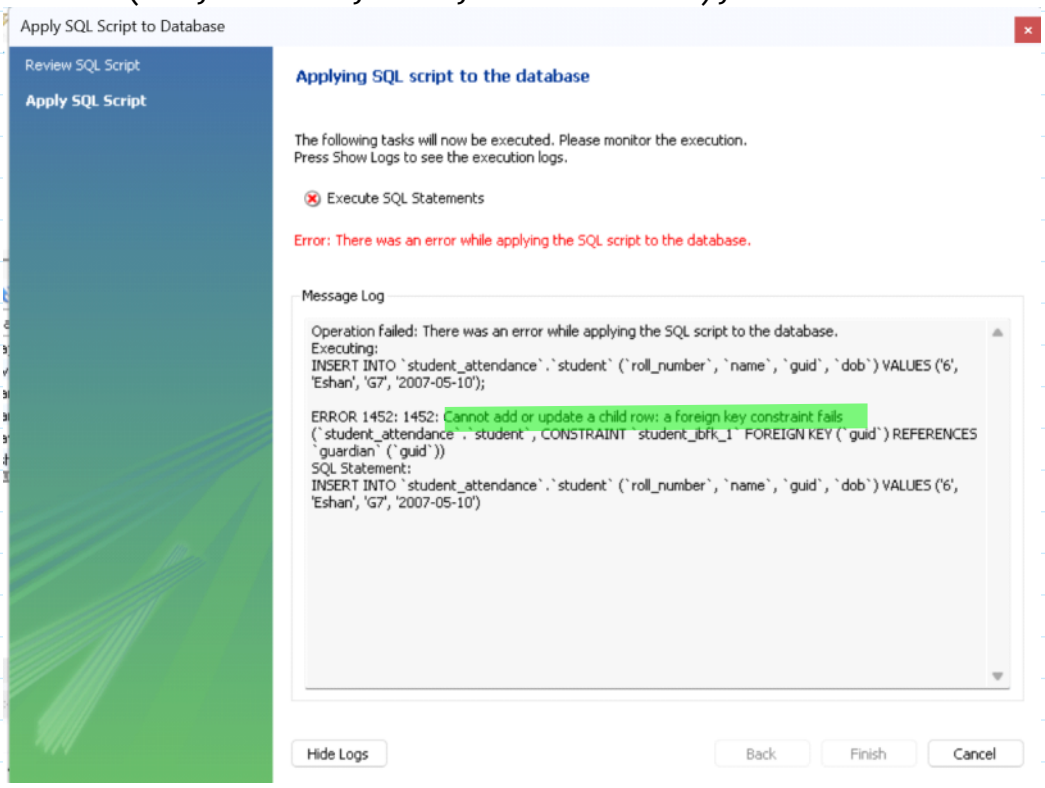
Syntax:

```
INSERT INTO table_name(colum1, colum2,…) VALUES(value1, value2, …);
```

```
mysql> INSERT INTO guardian(guid, name, mobile_number, address) VALUES
    -> ('G000000006', 'Sanjeev Dewat', 9885674327, 'Howrah')
    -> ;
Query OK, 1 row affected (0.467 sec)

mysql> select * from guardian;
+------------+----------------------+---------------+-----------+
| guid       | name                 | mobile_number | address   |
+------------+----------------------+---------------+-----------+
| G000000001 | Rakesh Jhunjhunwalla | 987678651     | Salt Lake |
| G000000002 | Mohit Singhi         | 987679871     | Salt Lake |
| G000000003 | Himanshu Agarwal     | 987670081     | Salt Lake |
| G000000004 | Amit Ahuja           | 988670081     | Salt Lake |
| G000000005 | Rajesh Singh         | 988670099     | Salt Lake |
| G000000006 | Sanjeev Dewat        | 9885674327    | Howrah    |
+------------+----------------------+---------------+-----------+
6 rows in set (0.025 sec)
```

```
INSERT INTO `student_attendance`.`student` (`roll_number`, `name`, `guid`, `dob`)
VALUES ('6', 'Eshan', 'G7', '2007-05-10');
```



SQL for DATA QUERY

So far we have learned how to create a database and how to store and manipulate data in them. We are interested in storing data in a database as it is easier to retrieve data in future from databases in whatever way we want. SQL provides efficient mechanisms to retrieve data stored in multiple tables in Mysql database. (or any other RDBMS) The SQL statement SELECT is used to retrieve data from the tables in a database and is also called a query statement.

SELECT statement

The SQL statement SELECT is used to retrieve data from the tables in a database and the output is also displayed in tabular format.

Syntax:

```
SELECT attribute1, attribute2, …
FROM table_name
WHERE condition;
```

Here, attribute1, Attribute2... are the column names of table table_name from which we want to retrieve data. The FROM clause is always written with SELECT clause as it specifies the name of the table from which data is to be retrieved. The WHERE clause is optional and is used to retrieve data that meets the specified condition(s).

To select all the data available in the table we use the following select statement:

```
SELECT * FROM table_name;
```

The following query retrieves the name and date of birth of a student with role number one:

```
mysql> SELECT name, dob FROM student
    -> WHERE roll_number = 1;
+------+------------+
| name | dob        |
+------+------------+
| Raj  | 2007-05-17 |
+------+------------+
1 row in set (0.149 sec)
```

```
CREATE TABLE `student_attendance`.`employee` (
  `EMPNO` INT NOT NULL,
  `ENAME` VARCHAR(45) NULL,
  `SALARY` INT NULL,
  `BONUS` INT NULL,
  `DEPTID` CHAR(4) NULL,
  PRIMARY KEY (`EMPNO`));


CREATE TABLE `student_attendance`.`department` (
  `depid` INT NOT NULL,
  `dept_name` VARCHAR(45) NULL,
  `city` VARCHAR(45) NULL,
  PRIMARY KEY (`depid`));

ALTER TABLE `student_attendance`.`department`
CHANGE COLUMN `depid` `depid` CHAR(4) NOT NULL ;


ALTER TABLE `student_attendance`.`employee`
ADD INDEX `depid_fk_idx` (`DEPTID` ASC) VISIBLE;
;
ALTER TABLE `student_attendance`.`employee`
ADD CONSTRAINT `depid_fk`
  FOREIGN KEY (`DEPTID`)
```

```
  REFERENCES `student_attendance`.`department` (`depid`)
  ON DELETE NO ACTION
  ON UPDATE NO ACTION;


INSERT INTO `student_attendance`.`department` (`depid`, `dept_name`, `city`) VALUES
('D001', 'IT', 'London');
INSERT INTO `student_attendance`.`department` (`depid`, `dept_name`, `city`) VALUES
('D002', 'Cyber Security', 'London');
INSERT INTO `student_attendance`.`department` (`depid`, `dept_name`, `city`) VALUES
('D003', 'Data Administration', 'London');
INSERT INTO `student_attendance`.`department` (`depid`, `dept_name`, `city`) VALUES
('D004', 'Software Research', 'San Jose');
INSERT INTO `student_attendance`.`department` (`depid`, `dept_name`, `city`) VALUES
('D005', 'Application Development', 'San Diego');


INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1001', 'Ravi', '10000', 'D001');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1002', 'Mukesh', '12000', 'D001');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1003', 'Jack', '15000', 'D001');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1004', 'Mark', '16000', 'D002');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1005', 'Allen', '20000', 'D003');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1006', 'Wiess', '18000', 'D002');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1007', 'Robert', '21000', 'D005');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1008', 'Kritka', '18000', 'D005');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1009', 'Joseph', '14000', 'D001');
INSERT INTO `student_attendance`.`employee` (`EMPNO`, `ENAME`, `SALARY`, `DEPTID`)
VALUES ('1010', 'Tanya', '15000', 'D002');
```

```
mysql> select * from department;
+-------+-------------------------+-----------+
| depid | dept_name               | city      |
+-------+-------------------------+-----------+
| D001  | IT                      | London    |
| D002  | Cyber Security          | London    |
| D003  | Data Administration     | London    |
| D004  | Software Research       | San Jose  |
| D005  | Application Development | San Diego |
+-------+-------------------------+-----------+
5 rows in set (0.138 sec)

mysql> select * from employee;
+-------+--------+--------+-------+--------+
| EMPNO | ENAME  | SALARY | BONUS | DEPTID |
+-------+--------+--------+-------+--------+
|  1001 | Ravi   |  10000 |  NULL | D001   |
|  1002 | Mukesh |  12000 |  NULL | D001   |
|  1003 | Jack   |  15000 |  NULL | D001   |
|  1004 | Mark   |  16000 |  NULL | D002   |
|  1005 | Allen  |  20000 |  NULL | D003   |
|  1006 | Wiess  |  18000 |  NULL | D002   |
|  1007 | Robert |  21000 |  NULL | D005   |
|  1008 | Kritka |  18000 |  NULL | D005   |
|  1009 | Joseph |  14000 |  NULL | D001   |
|  1010 | Tanya  |  15000 |  NULL | D002   |
+-------+--------+--------+-------+--------+
10 rows in set (0.035 sec)
```